Métriques IA

Évaluation des Modèles en Intelligence Artificielle

Régression • Classification • Computer Vision
NLP / LLM • RAG • AI Agents

Formules · Utilisation · Optimisation · Exemples

April 10, 2026

Contents

1 Régression **3**
1.1 MAE — Erreur Absolue Moyenne . . . 3
1.2 MSE — Erreur Quadratique Moyenne . . . 4
1.3 RMSE — Racine de l'Erreur Quadratique . . . 4
1.4 R^2 — Coefficient de Détermination . . . 5
1.5 MAPE — Erreur Absolue en Pourcentage . . . 6

2 Classification **7**
2.1 Accuracy — Exactitude . . . 7
2.2 Precision et Recall . . . 8
2.3 F1-Score . . . 8
2.4 MCC — Matthews Correlation Coefficient . . . 9
2.5 AUC-ROC . . . 9
2.6 Log Loss (Cross-Entropie) . . . 10

3 Computer Vision **12**
3.1 IoU — Intersection over Union . . . 12
3.2 mAP — mean Average Precision . . . 13
3.3 mIoU — Mean Intersection over Union (Segmentation) . . . 13
3.4 PSNR et SSIM (Qualité d'image) . . . 14

4 NLP et LLM **15**
4.1 Perplexité . . . 15
4.2 BLEU — Bilingual Evaluation Understudy . . . 16
4.3 ROUGE — Recall-Oriented Understudy for Gisting Evaluation . . . 16
4.4 BERTScore . . . 17
4.5 Pass@k — Génération de code . . . 18

5 RAG — Retrieval-Augmented Generation **19**
5.1 Hit Rate @k . . . 19
5.2 MRR — Mean Reciprocal Rank . . . 20
5.3 Faithfulness (RAGAS) . . . 20
5.4 Answer Relevancy (RAGAS) . . . 21
5.5 Context Precision et Context Recall . . . 22

6 AI Agents **23**
6.1 Task Success Rate . . . 23
6.2 Step Efficiency (Trajectory Optimality) . . . 24
6.3 Tool Precision . . . 24
6.4 Hallucination Rate & Scope Creep . . . 25

7 Deep Learning **26**
7.1 Fonctions de perte — Loss Functions . . . 26
7.1.1 Binary Cross-Entropy (BCE) . . . 26
7.1.2 Categorical Cross-Entropy . . . 27
7.1.3 Focal Loss . . . 27
7.1.4 Huber Loss (Smooth L1) . . . 28
7.1.5 KL Divergence & ELBO (VAE) . . . 29
7.2 Métriques d'entraînement . . . 29
7.2.1 Learning Rate & Gradient Norm . . . 29
7.2.2 Train Loss vs Val Loss — Diagnostic d'overfitting . . . 30
7.2.3 Batch Size & Throughput . . . 31
7.3 Métriques spécifiques aux architectures . . . 32
7.3.1 Réseaux convolutifs — Paramètres et Réceptivité . . . 32
7.3.2 Transformers — Attention et Complexité . . . 32
7.3.3 GAN — Métriques de qualité générative . . . 33

8 Tableau de Bord — Synthèse Générale **35**

Chapitre 1

Régression

La régression prédit une valeur numérique continue à partir de variables d'entrée. Ses métriques mesurent l'écart entre les valeurs prédites $\hat{y}_i$ et les vraies valeurs y_i.

1.1 MAE — Erreur Absolue Moyenne

Formule

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Quand utiliser

Quand les erreurs doivent être interprétées dans la même unité que y. Robuste aux valeurs aberrantes car elle ne les amplifie pas quadratiquement.

Comment optimiser

- Utiliser la régression **Quantile** ou la perte **Huber** pour réduire l'impact des outliers.
- Normaliser les features (StandardScaler, MinMaxScaler).
- Choisir des modèles robustes : **Gradient Boosting**, **Random Forest**.

Exemple illustratif

Prédiction de prix immobilier (en milliers \$) :

$$y = [200, 250, 300], \quad \hat{y} = [210, 240, 310]$$

$$\text{MAE} = \frac{|200-210| + |250-240| + |300-310|}{3} = \frac{10+10+10}{3} = \mathbf{10\,k\$}$$

Interprétation : en moyenne, la prédiction dévie de 10 000\$ du prix réel.

1.2 MSE — Erreur Quadratique Moyenne

Formule

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Quand utiliser

Utilisée comme **fonction de perte** lors de l'entraînement car elle est différentiable partout. Pénalise fortement les grandes erreurs.

Comment optimiser

- Détecter et traiter les **outliers** (IQR, Z-score) avant l'entraînement.
- Régularisation **Ridge** (L_2) pour éviter l'overfitting.
- Optimiser le **learning rate** avec un scheduler cosinus ou cyclique.

Exemple illustratif

Même exemple que MAE :

$$\text{MSE} = \frac{10^2 + 10^2 + 10^2}{3} = \frac{300}{3} = \mathbf{100}$$

Notez : unité = (k\$)^2, difficile à interpréter $\Rightarrow$ préférer RMSE.

1.3 RMSE — Racine de l'Erreur Quadratique

Formule

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} = \sqrt{\text{MSE}}$$

Quand utiliser

Même usage que MSE mais dans la **même unité** que y. Métrique de reporting standard.

Comment optimiser

- Comparer RMSE vs MAE : si RMSE $\gg$ MAE, il y a des outliers importants.
- Utiliser **log-transformation** sur y si la distribution est asymétrique.
- Entraîner des **ensembles** (stacking, blending).

Exemple illustratif

$$\text{RMSE} = \sqrt{100} = \mathbf{10\,k\$}$$

Même valeur que MAE ici car toutes les erreurs sont égales. En pratique, RMSE $\geq$ MAE.

1.4 R^2 — Coefficient de Détermination

Formule

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$$

où $\bar{y} = \frac{1}{n}\sum y_i$ est la moyenne des vraies valeurs.

Quand utiliser

Mesure la **proportion de variance expliquée** par le modèle. $R^2 = 1$ = prédiction parfaite ; $R^2 = 0$ = modèle équivalent à la moyenne ; $R^2 < 0$ = pire que la moyenne.

Comment optimiser

- Ajouter des **features pertinentes** (ingénierie des variables).
- Surveiller le R^2 **ajusté** pour les modèles avec beaucoup de variables :

$$R^2_{\text{adj}} = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

- Valider par **cross-validation** pour détecter l'overfitting.

Exemple illustratif

$$\bar{y} = 250, \quad \text{SS}_{\text{res}} = 300, \quad \text{SS}_{\text{tot}} = (200-250)^2 + (250-250)^2 + (300-250)^2 = 5000$$

$$R^2 = 1 - \frac{300}{5000} = 1 - 0.06 = \mathbf{0.94}$$

Le modèle explique **94%** de la variance des prix.

1.5 MAPE — Erreur Absolue en Pourcentage

Formule

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (\%)$$

Quand utiliser

Utile quand on veut une erreur en **pourcentage**, facilement communicable aux non-techniciens. Attention : instable si $y_i \approx 0$.

Comment optimiser

- Si y_i peut être zéro, utiliser **SMAPE** (Symmetric MAPE) :

$$\text{SMAPE} = \frac{200}{n} \sum \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

- Exclure les valeurs nulles ou quasi-nulles du jeu de test.

Exemple illustratif

$$\text{MAPE} = \frac{100}{3} \left(\frac{10}{200} + \frac{10}{250} + \frac{10}{300} \right) = \frac{100}{3}(0.05 + 0.04 + 0.033) = \mathbf{4.1}\%$$

Le modèle se trompe en moyenne de 4.1% sur le prix.

Chapitre 2

Classification

La classification prédit une catégorie discrète. On part toujours de la **matrice de confusion** :

	Prédit Positif	**Prédit Négatif**
Réel Positif	TP (Vrai Positif)	FN (Faux Négatif)
Réel Négatif	FP (Faux Positif)	TN (Vrai Négatif)

2.1 Accuracy — Exactitude

Formule

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Quand utiliser

Valide uniquement quand les **classes sont équilibrées**. Trompeuse avec des données déséquilibrées (ex : 99% de classe majoritaire ⇒ un modèle naïf obtient 99%).

Comment optimiser

- Rééchantillonner : **SMOTE** (suréchantillonnage) ou under-sampling.
- Ajuster le **class_weight** dans l'algorithme.
- Coupler avec F1 ou MCC pour les données déséquilibrées.

Exemple illustratif

Détection de spam : sur 100 emails, 90 normaux et 10 spams. Le modèle prédit :

$$TP = 8,\ TN = 88,\ FP = 2,\ FN = 2$$

$$\text{Accuracy} = \frac{8 + 88}{100} = \mathbf{96}\%$$

Semble excellent, mais 2 spams passent inaperçus $\Rightarrow$ vérifier le Recall.

2.2 Precision et Recall

Formule

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Quand utiliser

- **Precision** : quand les faux positifs sont coûteux (ex : accusation à tort).
- **Recall** : quand les faux négatifs sont coûteux (ex : cancer non détecté).

Comment optimiser

- Ajuster le **seuil de décision** (default 0.5) selon le compromis désiré.
- Tracer la **courbe Precision-Recall** pour choisir le seuil optimal.
- Utiliser **Focal Loss** pour les classes rares.

Exemple illustratif

Reprise détection spam :

$$\text{Precision} = \frac{8}{8+2} = \mathbf{80\%} \qquad \text{Recall} = \frac{8}{8+2} = \mathbf{80\%}$$

Sur 10 spams détectés, 8 sont vrais. Sur 10 vrais spams, 8 sont trouvés.

2.3 F1-Score

Formule

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

Généralisation F_β pondérée :

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{\beta^2\,\text{Precision} + \text{Recall}}$$

Quand utiliser

Métrique de référence pour les **classes déséquilibrées**. $\beta > 1$ favorise le Recall (médical), $\beta < 1$ favorise la Precision (filtrage).

Comment optimiser

- Maximiser le F1 en cherchant le meilleur seuil sur la courbe PR.
- Multi-classes : calculer **macro-F1** (moyenne par classe) ou **weighted-F1** (pondéré par support).

Exemple illustratif

$$F_1 = 2 \cdot \frac{0.80 \times 0.80}{0.80 + 0.80} = \frac{2 \times 0.64}{1.60} = \mathbf{0.80}$$

Un $F_1 = 0.80$ indique un bon équilibre Precision/Recall.

2.4 MCC — Matthews Correlation Coefficient

Formule

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \in [-1, 1]$$

Quand utiliser

Considéré comme la **métrique la plus fiable** pour la classification binaire déséquilibrée. Prend en compte toutes les cases de la matrice de confusion.

Comment optimiser

- Privilégier le MCC à l'Accuracy sur tout dataset déséquilibré.
- Optimiser directement le MCC en ajustant les seuils de probabilité.

Exemple illustratif

$$\text{MCC} = \frac{8 \cdot 88 - 2 \cdot 2}{\sqrt{(8+2)(8+2)(88+2)(88+2)}} = \frac{704 - 4}{\sqrt{10 \cdot 10 \cdot 90 \cdot 90}} = \frac{700}{900} \approx \mathbf{0.78}$$

Valeur proche de 1 : excellent classifieur malgré le déséquilibre.

2.5 AUC-ROC

Formule

Courbe ROC : TPR vs FPR pour tous les seuils.

$$\text{TPR} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

$$\text{AUC} = \int_0^1 \text{TPR}(t)\, d\,\text{FPR}(t) \quad \in [0.5, 1]$$

Quand utiliser

Mesure la **performance globale** du classifieur indépendamment du seuil. AUC = 1 : parfait ; AUC = 0.5 : aléatoire.

Comment optimiser

- Utiliser **AUC-PR** (Precision-Recall) quand la classe positive est rare.
- Optimiser la **perte de rang** (pairwise ranking loss).
- Calibrer les probabilités avec **Platt scaling** ou **isotonic regression**.

Exemple illustratif

Un modèle AUC = 0.95 signifie : en prenant un exemple positif et un négatif au hasard, le modèle attribue une probabilité plus haute au positif dans 95% des cas.

2.6 Log Loss (Cross-Entropie)

Formule

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\right]$$

Multi-classes (K classes) :

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \log \hat{p}_{ik}$$

Quand utiliser

Mesure la qualité des **probabilités prédites** (pas seulement la classe). Utilisée comme fonction de perte pour la régression logistique et les réseaux de neurones.

Comment optimiser

- **Calibration** des probabilités pour réduire le log loss.
- Régularisation (L_1, L_2, Dropout) pour éviter les probabilités extrêmes.
- **Label smoothing** : remplacer 0/1 par ϵ / $1 - \epsilon$.

Exemple illustratif

Deux emails : vrai spam ($y = 1$) prédit à $\hat{p} = 0.9$, non-spam ($y = 0$) prédit à $\hat{p} = 0.1$:

$$\mathcal{L} = -\frac{1}{2}\left[\log(0.9) + \log(1 - 0.1)\right] = -\frac{1}{2}(-0.105 - 0.105) = \mathbf{0.105}$$

Chapitre 3

Computer Vision

3.1 IoU — Intersection over Union

Formule

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} = \frac{\text{Aire intersection}}{\text{Aire union}}$$

Quand utiliser

Base de toutes les métriques de **détection d'objets**. IoU $\geq$ 0.5 : True Positive (convention PASCAL VOC).

Comment optimiser

- Utiliser **DIoU** / **CIoU** comme loss pour mieux guider la régression des boîtes.
- Affiner le seuil NMS (Non-Maximum Suppression) : 0.45-0.6.
- Augmenter avec **mosaic, mixup, random crop**.

Exemple illustratif

Boîte prédite : $[10, 10, 50, 50]$, boîte GT : $[20, 20, 60, 60]$. Intersection : $[20, 20, 50, 50] = 30 \times 30 = 900$. Union : $40^2 + 40^2 - 900 = 2300$.

$$\text{IoU} = \frac{900}{2300} \approx \mathbf{0.39}$$

IoU $< 0.5 \Rightarrow$ considéré comme **Faux Positif** au seuil PASCAL VOC.

3.2 mAP — mean Average Precision

Formule

$$\mathrm{AP}_c = \int_0^1 p_c(r)\,dr \approx \sum_{k=1}^{N} p(k) \cdot \Delta r(k)$$

$$\mathrm{mAP} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c$$

Standard COCO : $\mathrm{mAP@[.5:.95]} = \frac{1}{10}\sum_{t=0.5}^{0.95} \mathrm{mAP}(t)$

Quand utiliser

Métrique principale pour la **détection d'objets** (YOLO, Faster R-CNN, DETR). Mesure la performance sur toutes les classes et tous les seuils IoU.

Comment optimiser

- Augmenter la **résolution d'entrée** (640 → 1280).
- Utiliser des **ancres optimisées** via k-means sur le dataset.
- **Test-Time Augmentation** (TTA) : flip, scale, crop.
- Multi-scale training (images de tailles variées).

Exemple illustratif

Détection de chats et chiens : $\mathrm{AP}_{\text{chat}} = 0.82$, $\mathrm{AP}_{\text{chien}} = 0.76$.

$$\mathrm{mAP} = \frac{0.82 + 0.76}{2} = \mathbf{0.79}$$

Le modèle détecte mieux les chats (82%) que les chiens (76%).

3.3 mIoU — Mean Intersection over Union (Segmentation)

Formule

$$\mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \frac{TP_c}{TP_c + FP_c + FN_c}$$

Quand utiliser

Métrique standard pour la **segmentation sémantique** et d'instance. Mesure l'overlap pixel-par-pixel pour chaque classe.

Comment optimiser

- Utiliser **DeepLabV3+**, **SegFormer** ou **SAM**.
- **Dilated convolutions** pour augmenter le champ réceptif.
- **CRF post-processing** pour affiner les bordures.

Exemple illustratif

Segmentation route/voiture/fond sur 1000 pixels :

$$\text{IoU}_{\text{route}} = 0.85, \quad \text{IoU}_{\text{voiture}} = 0.72, \quad \text{IoU}_{\text{fond}} = 0.91$$

$$\text{mIoU} = \frac{0.85 + 0.72 + 0.91}{3} = \mathbf{0.827}$$

3.4 PSNR et SSIM (Qualité d'image)

Formule

$$\text{PSNR} = 10 \log_{10}\left(\frac{MAX^2}{\text{MSE}}\right) \quad \text{(en dB)}$$

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Quand utiliser

GAN, super-résolution, compression. PSNR : basé sur le signal. SSIM : basé sur la **perception humaine** (structure, luminance, contraste).

Comment optimiser

- Combiner perte **perceptuelle** (VGG features) + perte pixel.
- Utiliser **LPIPS** (Learned Perceptual Image Patch Similarity) comme métrique complémentaire.

Exemple illustratif

Image 8 bits ($MAX = 255$), MSE $= 100$:

$$\text{PSNR} = 10 \log_{10}\left(\frac{255^2}{100}\right) = 10 \log_{10}(650.25) \approx \mathbf{28.1\,dB}$$

En pratique : > 40 dB excellent, $30-40$ dB bon, < 30 dB perceptible.

Chapitre 4

NLP et LLM

4.1 Perplexité

Formule

$$\mathrm{PPL}(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i \mid w_1, \ldots, w_{i-1})\right)$$

Quand utiliser

Mesure la **capacité de généralisation** d'un modèle de langage. Plus la perplexité est basse, meilleur est le modèle. Utilisée pour comparer des LLMs sur un corpus de référence.

Comment optimiser

- Augmenter la **taille du modèle** (loi d'échelle de Chinchilla).
- Entraîner sur plus de données de haute qualité (filtrage web).
- Utiliser **BPE/SentencePiece** pour un tokenizer adapté.
- **Sliding window** pour les longs contextes.

Exemple illustratif

Phrase : *"Le chat mange"* (3 tokens). Probabilités : $P(\mathrm{Le}) = 0.1$, $P(\mathrm{chat}|\mathrm{Le}) = 0.3$, $P(\mathrm{mange}|\cdots) = 0.2$.

$$\mathrm{PPL} = \exp\left(-\frac{\log(0.1) + \log(0.3) + \log(0.2)}{3}\right) = \exp(1.57) \approx \mathbf{4.8}$$

GPT-4 atteint PPL≈5-10 sur WikiText-103.

4.2 BLEU — Bilingual Evaluation Understudy

Formule

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

$$BP = \begin{cases} 1 & \text{si } c > r \\ e^{1-r/c} & \text{si } c \leq r \end{cases} \qquad p_n = \frac{\sum_{\text{ngram}\in\hat{y}} \min(\text{count}, \text{count}_{ref})}{\sum_{\text{ngram}\in\hat{y}} \text{count}}$$

où c = longueur candidate, r = longueur référence, p_n = précision n-gramme.

Quand utiliser

Standard historique pour la **traduction automatique**. BLEU-4 ($N = 4$, $w_n = 0.25$) est le plus courant. Limité : ne capture pas la sémantique.

Comment optimiser

- **Beam search** avec taille de faisceau 4-8 améliore le BLEU.
- Entraîner avec **REINFORCE** directement sur le BLEU (RLHF).
- Compléter avec **BERTScore** et **COMET** pour une évaluation sémantique.

Exemple illustratif

Référence : *"the cat is on the mat"*
Prédiction : *"the cat is on the mat"* $\Rightarrow$ BLEU = **1.0**
Prédiction : *"the cat sat on the rug"* $\Rightarrow$ BLEU $\approx$ **0.48**
(4/6 unigrammes corrects mais bigrammes et trigrammes dégradés)

4.3 ROUGE — Recall-Oriented Understudy for Gisting Evaluation

Formule

$$\text{ROUGE-N} = \frac{\sum_{\text{gram}_n \in \text{Ref}} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{\text{gram}_n \in \text{Ref}} \text{Count}(\text{gram}_n)}$$

$$\text{ROUGE-L} = \frac{(1+\beta^2) \cdot R_{lcs} \cdot P_{lcs}}{R_{lcs} + \beta^2 \cdot P_{lcs}}$$

où LCS = plus longue sous-séquence commune.

Quand utiliser

Standard pour l'**évaluation de résumés automatiques**. ROUGE-1 (unigrammes), ROUGE-2 (bigrammes), ROUGE-L (structure).

Comment optimiser

- Optimiser avec **extractive** + **abstractive** combinés.
- Utiliser des **modèles seq2seq** pré-entraînés (BART, T5, Pegasus).
- Coupler ROUGE avec **BERTScore** pour la qualité sémantique.

Exemple illustratif

Référence : *"le chat mange du poisson"* (5 mots)
Résumé prédit : *"le chat mange"* (3 mots)
ROUGE-1 $= \frac{3}{5} =$ **0.60** (3 unigrammes communs sur 5 dans la référence)

4.4 BERTScore

Formule

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \cos(e_{x_i}, e_{\hat{x}_j})$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} \cos(e_{\hat{x}_j}, e_{x_i})$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

Quand utiliser

Mesure la **similarité sémantique** entre texte prédit et référence via des embeddings contextuels. Corrèle mieux avec le jugement humain que BLEU/ROUGE.

Comment optimiser

- Choisir un encodeur de qualité : **DeBERTa**, **mE5**, **multilingual-e5**.
- Utiliser **IDF weighting** pour pondérer les tokens rares.

Exemple illustratif

Référence : *"voiture rapide"* Prédiction : *"automobile rapide"*
BLEU = 0.5 ("voiture" $\neq$ "automobile")
BERTScore $\approx$ **0.93** (les embeddings de "voiture" et "automobile" sont très proches)

4.5 Pass@k — Génération de code

Formule

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} = 1 - \frac{(n-c)!\,(n-k)!}{(n-c-k)!\,n!}$$

où n = tentatives générées, c = solutions correctes, k = tentatives autorisées.

Quand utiliser

Mesure la capacité à générer du **code fonctionnel**. Benchmark HumanEval (164 problèmes Python). pass@1 = une seule tentative autorisée.

Comment optimiser

- Fine-tuning sur **code de haute qualité** (GitHub stars élevés).
- **RLHF sur les tests unitaires** (CodeRL, PPOCoder).
- Augmenter k avec **self-consistency** : voter parmi k solutions.

Exemple illustratif

On génère $n = 10$ solutions, $c = 7$ passent les tests, $k = 3$:

$$\text{pass@3} = 1 - \frac{\binom{3}{3}}{\binom{10}{3}} = 1 - \frac{1}{120} \approx \mathbf{0.992}$$

En 3 tentatives, on a quasi-certitude d'avoir une solution correcte.

Chapitre 5

RAG — Retrieval-Augmented Generation

5.1 Hit Rate @k

Formule

$$\text{HitRate@k} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}\left[\exists\, d \in \mathcal{R}_k(q) : d \in \mathcal{D}_q^*\right]$$

où $\mathcal{R}_k(q)$ = top-k chunks récupérés, $\mathcal{D}_q^*$ = documents pertinents.

Quand utiliser

Première métrique à vérifier pour le composant **retrieval**. Si Hit Rate@5 est bas, le LLM ne pourra jamais répondre correctement.

Comment optimiser

- Tester différents **chunking** : taille, overlap, sémantique.
- Utiliser **Hybrid Search** (BM25 + dense) avec RRF (Reciprocal Rank Fusion).
- Affiner le **modèle d'embedding** sur le domaine métier.
- Appliquer un **reranker** cross-encoder en post-retrieval.

Exemple illustratif

Sur 10 questions, 8 trouvent le bon chunk dans le top-5 :

$$\text{HitRate@5} = \frac{8}{10} = \mathbf{0.80}$$

Objectif production : Hit Rate@5 ≥ 0.85.

5.2 MRR — Mean Reciprocal Rank

Formule

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q}$$

où rank_q = position du premier document pertinent pour la question q.

Quand utiliser

Mesure si le **meilleur document** est en première position. Idéal pour les systèmes QA où le premier résultat est utilisé en priorité.

Comment optimiser

- **Query rewriting** : reformuler la question pour mieux matcher les chunks.
- **HyDE** (Hypothetical Document Embeddings) : générer un document hypothétique puis retriever.
- Fine-tuner l'embedder avec **paires (question, passage)** du domaine.

Exemple illustratif

3 questions, le bon chunk est en position 1, 3, 2 :

$$\text{MRR} = \frac{1}{3}\left(\frac{1}{1} + \frac{1}{3} + \frac{1}{2}\right) = \frac{1}{3}(1 + 0.33 + 0.5) = \mathbf{0.61}$$

5.3 Faithfulness (RAGAS)

Formule

$$\text{Faithfulness} = \frac{|\text{claims supportés par le contexte}|}{|\text{total claims dans la réponse}|}$$

Quand utiliser

Mesure les **hallucinations** : est-ce que chaque affirmation de la réponse est bien fondée sur le contexte récupéré ? Métrique critique en production.

Comment optimiser

- Ajouter une **instruction système** : "Réponds uniquement à partir du contexte fourni".
- Utiliser **Corrective RAG** : re-retriever si le contexte est jugé insuffisant.

- Réduire la **température** du LLM (0.0-0.3 en RAG).
- Implémenter un **grounding checker** post-génération.

Exemple illustratif

Réponse du LLM contient 5 affirmations. Le contexte en supporte 4. Une est inventée.

$$\text{Faithfulness} = \frac{4}{5} = \mathbf{0.80}$$

Objectif : ≥ 0.90 en production.

5.4 Answer Relevancy (RAGAS)

Formule

$$\text{AnswerRelevancy} = \frac{1}{N}\sum_{i=1}^{N} \cos\big(E(q_i^{\text{généré}}),\ E(q^{\text{original}})\big)$$

où $q_i^{\text{généré}}$ = questions générées à partir de la réponse, E = embeddings.

Quand utiliser

Détecte les réponses **hors-sujet** ou trop génériques. Une réponse doit répondre à la vraie question, pas paraphraser le contexte.

Comment optimiser

- Améliorer le **prompt template** : inclure la question explicitement.
- Utiliser un LLM plus capable en **instruction-following**.
- Post-filtrage : vérifier que la réponse contient des mots-clés de la question.

Exemple illustratif

Question : *"Quelle est la capitale de la France ?"*
Réponse A : *"Paris est la capitale"* $\Rightarrow$ Relevancy $\approx$ **0.97**
Réponse B : *"La France est un pays européen riche en histoire"* $\Rightarrow$ Relevancy $\approx$ **0.41**

5.5 Context Precision et Context Recall

Formule

$$\text{ContextPrecision@k} = \frac{\sum_{k=1}^{K} \text{Precision@k} \cdot v_k}{\text{nombre de chunks pertinents dans top-k}}$$

$$\text{ContextRecall} = \frac{|\text{claims de la GT couverts par le contexte}|}{|\text{total claims dans la GT}|}$$

Quand utiliser

Context Precision : le contexte récupéré est-il utile ? **Context Recall** : manque-t-il des informations importantes dans le contexte ?

Comment optimiser

- Precision faible ⇒ améliorer la **pertinence du reranker**.
- Recall faible ⇒ augmenter k ou améliorer **l'index** (coverage).
- **Parent-child chunking** pour équilibrer les deux.

Exemple illustratif

Top-3 chunks : [pertinent, non-pertinent, pertinent]. GT a 3 claims, 2 couverts.

$$\text{ContextPrecision@3} \approx \mathbf{0.56} \qquad \text{ContextRecall} = \frac{2}{3} \approx \mathbf{0.67}$$

Chapitre 6

AI Agents

6.1 Task Success Rate

Formule

$$\text{TSR} = \frac{N_{\text{tâches réussies}}}{N_{\text{total tâches}}}$$

Pour les tâches multi-objectifs :

$$\text{TSR}_{\text{partiel}} = \frac{1}{N} \sum_{i=1}^{N} \frac{\text{sous-objectifs réussis}_i}{\text{total sous-objectifs}_i}$$

Quand utiliser

Métrique principale des benchmarks agents (**WebArena, GAIA, AgentBench**). Mesure si l'agent accomplit la tâche de bout en bout.

Comment optimiser

- Améliorer le **prompting système** avec des exemples few-shot.
- Implémenter la **réflexion** : l'agent vérifie son propre travail (Reflexion).
- Décomposer les tâches complexes en **sous-tâches** (Plan & Execute).
- Utiliser des **outils de vérification** (linters, tests unitaires pour le code).

Exemple illustratif

Agent de réservation : 20 tâches testées, 14 complétées avec succès.

$$\text{TSR} = \frac{14}{20} = \mathbf{70}\%$$

GPT-4 atteint ≈45% sur WebArena, les meilleurs agents ≈55-60%.

6.2 Step Efficiency (Trajectory Optimality)

Formule

$$\text{StepEfficiency} = \frac{T_{\text{optimal}}}{T_{\text{réel}}} \leq 1$$

$$\text{TrajectoryAccuracy} = \frac{|\text{steps corrects}|}{|\text{total steps}|}$$

Quand utiliser

Mesure l'**efficacité du chemin** emprunté. Un agent peut réussir la tâche mais faire 3× plus d'étapes que nécessaire.

Comment optimiser

- Définir un **budget de steps** (max_iterations) pour forcer l'efficacité.
- Entraîner avec **RLHF** en pénalisant les trajectoires longues.
- Utiliser **Tree of Thoughts** avec pruning pour explorer moins de chemins.

Exemple illustratif

Tâche : réserver un vol. Chemin optimal : 5 étapes. L'agent en fait 12.

$$\text{StepEfficiency} = \frac{5}{12} \approx \mathbf{0.42}$$

L'agent réussit mais avec une efficacité faible ⇒ revoir la stratégie de planification.

6.3 Tool Precision

Formule

$$\text{ToolPrecision} = \frac{N_{\text{appels outils utiles}}}{N_{\text{appels outils total}}}$$

$$\text{ToolRecall} = \frac{N_{\text{outils nécessaires appelés}}}{N_{\text{outils nécessaires}}}$$

Quand utiliser

Mesure si l'agent **sélectionne les bons outils** et les utilise à bon escient. Un agent qui appelle des outils inutilement consomme des tokens et du temps.

Comment optimiser

- Améliorer les **descriptions des outils** (claires, avec exemples d'usage).
- Utiliser **MCP** (Model Context Protocol) pour une découverte dynamique.
- Implémenter du **tool caching** pour éviter les appels redondants.

Exemple illustratif

Agent effectue 10 appels d'outils : 7 utiles, 3 inutiles. 2 outils nécessaires oubliés sur 9.

$$\text{ToolPrecision} = \frac{7}{10} = \mathbf{0.70} \qquad \text{ToolRecall} = \frac{7}{9} \approx \mathbf{0.78}$$

6.4 Hallucination Rate & Scope Creep

Formule

$$\text{HallucinationRate} = \frac{N_{\text{actions basées sur faits inventés}}}{N_{\text{total actions}}}$$

$$\text{ScopeCreep} = \frac{N_{\text{actions hors périmètre}}}{N_{\text{total actions}}}$$

Quand utiliser

Métriques de **sécurité et fiabilité**. Critiques pour les agents autonomes en production (agents financiers, médicaux, administratifs).

Comment optimiser

- **Grounding obligatoire** : forcer l'agent à citer ses sources pour chaque action.
- **Sandboxing** : limiter les permissions des outils (lecture seule d'abord).
- Implémenter un **guardrail LLM** qui valide chaque action avant exécution.
- **Human-in-the-loop** pour les actions irréversibles (envoi d'emails, paiements).

Exemple illustratif

Agent de support client : sur 50 actions, 3 inventent des politiques inexistantes (hallucination), 2 accèdent à des données non autorisées (scope creep).

$$\text{HallucinationRate} = \frac{3}{50} = \mathbf{6\%} \qquad \text{ScopeCreep} = \frac{2}{50} = \mathbf{4\%}$$

Objectif production : $< 1\%$ pour chacune.

Chapitre 7

Deep Learning

Le Deep Learning introduit des métriques propres à l'entraînement des réseaux de neurones : fonctions de perte, régularisation, et comportement dynamique pendant la descente de gradient.

7.1 Fonctions de perte — Loss Functions

7.1.1 Binary Cross-Entropy (BCE)

Formule

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)\right]$$

Quand utiliser

Classification binaire avec activation **Sigmoid** en sortie. Pénalise fortement les prédictions confiantes et erronées.

Comment optimiser

- Utiliser **BCEWithLogitsLoss** (PyTorch) pour plus de stabilité numérique.
- Ajouter **pos_weight** pour gérer les classes déséquilibrées.
- **Label smoothing** : remplacer $y \in \{0, 1\}$ par $y \in \{\epsilon, 1-\epsilon\}$.

Exemple illustratif

$y = 1$ (positif réel), $\hat{p} = 0.9$ (prédit avec confiance) :

$$\mathcal{L} = -[1 \cdot \log(0.9) + 0] = -(-0.105) = \mathbf{0.105}$$

$y = 1$, $\hat{p} = 0.1$ (erreur confiante) :

$$\mathcal{L} = -\log(0.1) = \mathbf{2.303} \quad \text{(pénalité } 22\times \text{ plus élevée)}$$

7.1.2 Categorical Cross-Entropy

Formule

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(\hat{p}_{ik})$$

avec activation **Softmax** : $\hat{p}_{ik} = \dfrac{e^{z_{ik}}}{\sum_j e^{z_{ij}}}$

Quand utiliser

Classification multi-classes ($K > 2$). Standard pour les réseaux convolutifs (ResNet, VGG) et les transformers de classification.

Comment optimiser

- **Temperature scaling** : diviser les logits par $T > 1$ pour calibrer les probabilités.
- **Mixup** : interpoler deux exemples et leurs labels pour régulariser.
- **CutMix** : découper et coller des patches d'images avec labels mélangés.

Exemple illustratif

3 classes (chat, chien, oiseau). Vrai label : chat ($y = [1, 0, 0]$). Softmax prédit $\hat{p} = [0.7, 0.2, 0.1]$:

$$\mathcal{L} = -(1 \cdot \log 0.7 + 0 + 0) = -(-0.357) = \mathbf{0.357}$$

7.1.3 Focal Loss

Formule

$$\mathcal{L}_{\text{FL}} = -\frac{1}{n}\sum_{i=1}^{n} \alpha_i(1-\hat{p}_i)^{\gamma}\log(\hat{p}_i)$$

avec $\gamma \geq 0$ (facteur de focus, typiquement $\gamma = 2$) et α poids de classe.

Quand utiliser

Conçue pour la **détection d'objets** (RetinaNet) et les datasets très déséquilibrés. Le facteur $(1-\hat{p})^{\gamma}$ réduit la contribution des exemples faciles.

Comment optimiser

- Chercher γ par grid search : typiquement $\gamma \in \{0.5, 1, 2, 5\}$.
- Combiner avec **class balancing** (α inversement proportionnel à la fréquence).

- Remplace avantageusement BCE sur tout dataset déséquilibré (>10:1).

Exemple illustratif

Exemple facile ($\hat{p} = 0.9$, $\gamma = 2$) vs exemple difficile ($\hat{p} = 0.3$, $\gamma = 2$) :

$$\mathcal{L}_{\text{facile}} = -(1-0.9)^2 \log(0.9) = 0.01 \times 0.105 = \mathbf{0.001}$$

$$\mathcal{L}_{\text{difficile}} = -(1-0.3)^2 \log(0.3) = 0.49 \times 1.204 = \mathbf{0.590}$$

L'exemple difficile reçoit 590× plus d'attention.

7.1.4 Huber Loss (Smooth L1)

Formule

$$\mathcal{L}_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{si } |y-\hat{y}| \leq \delta \\ \delta\left(|y-\hat{y}| - \frac{\delta}{2}\right) & \text{sinon} \end{cases}$$

Quand utiliser

Régression robuste aux outliers. Utilisée dans **DQN** (Q-learning), détection d'objets (régression des boîtes), et partout où MSE est trop sensible aux valeurs aberrantes.

Comment optimiser

- Choisir δ selon l'échelle des erreurs attendues (souvent $\delta = 1$).
- Préférer **Log-Cosh Loss** si une différentiabilité complète est requise :

$$\mathcal{L} = \sum \log\cosh(\hat{y}_i - y_i)$$

Exemple illustratif

$\delta = 1$, erreur $e = 0.5$ (petite) : $\mathcal{L} = \frac{1}{2}(0.5)^2 = \mathbf{0.125}$ (comportement MSE)
Erreur $e = 3$ (grande outlier) : $\mathcal{L} = 1 \times (3 - 0.5) = \mathbf{2.5}$ (comportement MAE, pas 4.5 comme MSE)

7.1.5 KL Divergence & ELBO (VAE)

Formule

$$D_{\mathrm{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_P\left[\log \frac{P(x)}{Q(x)}\right] \geq 0$$

ELBO (Evidence Lower BOund) pour VAE :

$$\mathcal{L}_{\mathrm{VAE}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z|x)\|p(z))}_{\text{régularisation latente}}$$

Quand utiliser

VAE (Variational Autoencoders), distillation de connaissances, alignement de distributions (RLHF avec KL penalty sur la politique).

Comment optimiser

- β**-VAE** : multiplier le terme KL par $\beta > 1$ pour un espace latent plus disentanglé.
- **Free bits** : garantir un minimum de KL par dimension latente.
- Anneal le poids KL de 0 à 1 pendant l'entraînement (**KL annealing**).

Exemple illustratif

$P = [0.4, 0.6]$, $Q = [0.5, 0.5]$:

$$D_{\mathrm{KL}}(P\|Q) = 0.4 \log \frac{0.4}{0.5} + 0.6 \log \frac{0.6}{0.5} = 0.4(-0.223) + 0.6(0.182) = \mathbf{0.020}$$

Distributions proches $\Rightarrow$ KL faible. Si $P = Q$, alors $D_{\mathrm{KL}} = 0$.

7.2 Métriques d'entraînement

7.2.1 Learning Rate & Gradient Norm

Formule

Mise à jour SGD avec momentum :

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta \mathcal{L} \qquad \theta \leftarrow \theta - \eta\, v_t$$

Adam (Adaptive Moment Estimation) :

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

Gradient norm (suivi pendant l'entraînement) :

$$\|\nabla\| = \sqrt{\sum_l \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} \right)^2}$$

Quand utiliser

Surveiller $\|\nabla\|$ détecte l'**explosion** ($\|\nabla\| \gg 1$) ou la **disparition** ($\|\nabla\| \approx 0$) du gradient. Le learning rate est le hyperparamètre le plus critique.

Comment optimiser

- **Gradient clipping** : $\nabla \leftarrow \nabla \cdot \min(1, c/\|\nabla\|)$ avec $c = 1$ ou $c = 5$.
- **LR Warmup** + cosine decay : monter lentement puis descendre en cosinus.
- **LR Finder** (fast.ai) : identifier le LR optimal avant d'entraîner.
- **AdamW** (Adam + weight decay découplé) : standard pour les transformers.

Exemple illustratif

Adam avec $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$: au step $t = 1$, $\nabla = 0.1$:

$$m_1 = 0.1(1 - 0.9) = 0.01, \quad v_1 = 0.01(1 - 0.999) = 10^{-5}$$

$$\theta \leftarrow \theta - \frac{10^{-3}}{\sqrt{10^{-5}} + 10^{-8}} \times 0.1 \approx \theta - \mathbf{0.316}$$

7.2.2 Train Loss vs Val Loss — Diagnostic d'overfitting

Formule

$$\text{Overfitting Gap} = \mathcal{L}_{\text{val}} - \mathcal{L}_{\text{train}} > 0$$

$$\text{Generalization Error} = \mathbb{E}[\mathcal{L}_{\text{test}}] - \mathbb{E}[\mathcal{L}_{\text{train}}]$$

Quand utiliser

Diagnostic fondamental en DL. Courbes d'apprentissage à surveiller à chaque epoch via **TensorBoard** ou **WandB**.

Comment optimiser

- **Dropout** ($p = 0.1$–0.5) : désactiver aléatoirement des neurones.
- **Weight Decay** ($\lambda = 10^{-4}$–10^{-2}) : pénalité L_2 sur les poids.

- **Early Stopping** : arrêter dès que $\mathcal{L}_{\text{val}}$ ne diminue plus (patience = 5–10).
- **Data Augmentation** : augmenter la diversité des données d'entraînement.
- **Batch Normalization** / **Layer Normalization** pour stabiliser.

Exemple illustratif

Epoch 50 : $\mathcal{L}_{\text{train}} = 0.05$, $\mathcal{L}_{\text{val}} = 0.42$.
Gap $= 0.42 - 0.05 = \mathbf{0.37} \Rightarrow$ overfitting sévère.
Action : ajouter Dropout(0.3) + Weight Decay(10^{-3}) + augmentation.

7.2.3 Batch Size & Throughput

Formule

$$\text{Throughput} = \frac{N_{\text{samples}}}{t_{\text{epoch}}} \quad (\text{samples/sec})$$

$$\text{GPU Utilization} = \frac{t_{\text{calcul}}}{t_{\text{total}}} \times 100\%$$

$$\text{MFU} = \frac{\text{FLOPs réels}}{\text{FLOPs théoriques GPU}}$$

Quand utiliser

Métriques système critiques pour l'**optimisation de l'entraînement** et la réduction des coûts cloud (GPU H100, A100). MFU est le KPI clé pour les LLMs.

Comment optimiser

- **Mixed Precision** (fp16/bf16) : 2× throughput, 2× moins de VRAM.
- **Gradient Accumulation** : simuler un grand batch avec peu de VRAM.
- **Flash Attention** : attention en $O(N)$ mémoire au lieu de $O(N^2)$.
- **torch.compile** (PyTorch 2.0) : fusion des opérations pour +20-30% throughput.
- **DataLoader** : `num_workers`≥4, `pin_memory=True`.

Exemple illustratif

GPT-3 (175B params) entraîné sur A100 80GB :

$$\text{FLOPs/token} \approx 2 \times 175 \times 10^9 = 3.5 \times 10^{11}$$

$$\text{MFU} = \frac{3.5 \times 10^{11} \times 250 \text{ tok/s}}{312 \times 10^{12} \text{ FLOP/s}} \approx \mathbf{28\%}$$

Objectif : MFU $\geq 40\%$ (bon entraînement).

7.3 Métriques spécifiques aux architectures

7.3.1 Réseaux convolutifs — Paramètres et Réceptivité

Formule

Nombre de paramètres d'une couche Conv2D :

$$N_{\text{params}} = (k_h \times k_w \times C_{\text{in}} + 1) \times C_{\text{out}}$$

Champ réceptif effectif (Effective Receptive Field) :

$$r_l = r_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

où k_l = taille du kernel, s_i = stride à la couche i.

Quand utiliser

Concevoir des architectures CNN efficaces (ResNet, EfficientNet). Le champ réceptif doit couvrir les objets à détecter.

Comment optimiser

- **Depthwise Separable Conv** : diviser les params par $\approx C_{\text{in}}$ (MobileNet).
- **Dilated convolutions** : augmenter le champ réceptif sans plus de params.
- **EfficientNet scaling** : augmenter profondeur, largeur et résolution conjointement.

Exemple illustratif

Conv2D : $k = 3$, $C_{\text{in}} = 64$, $C_{\text{out}} = 128$:

$$N_{\text{params}} = (3 \times 3 \times 64 + 1) \times 128 = 577 \times 128 = \mathbf{73\,856}$$

Champ réceptif après 5 couches Conv 3×3 (stride 1) :

$$r = 1 + 4 \times (3 - 1) = \mathbf{9}$$

7.3.2 Transformers — Attention et Complexité

Formule

Scaled Dot-Product Attention :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Complexité : $O(n^2 \cdot d)$ en temps et mémoire (n = longueur séquence, d = dimension).

Multi-Head Attention :

$$\mathrm{MHA}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)W^O$$

$$\text{où } \mathrm{head}_i = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Quand utiliser

Architecture de base de tous les LLMs (GPT, BERT, T5, Llama). La complexité quadratique $O(n^2)$ est le goulot d'étranglement pour les longs contextes.

Comment optimiser

- **Flash Attention 2/3** : $O(n)$ mémoire via tiling sur HBM.
- **GQA** (Grouped Query Attention) : partager les têtes K,V (Llama 3).
- **RoPE** (Rotary Position Embedding) : meilleures performances sur long contexte.
- **Sliding Window Attention** (Mistral) : attention locale + globale.

Exemple illustratif

GPT-2 Small : $n = 1024$, $d_k = 64$, $h = 12$ têtes.
FLOPs attention par couche :

$$\approx 4 \times n^2 \times d = 4 \times 1024^2 \times 768 \approx \mathbf{3.2 \times 10^9}$$

Avec Flash Attention : même résultat, 10× moins de mémoire utilisée.

7.3.3 GAN — Métriques de qualité générative

Formule

Perte GAN originale (minimax) :

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

FID (Fréchet Inception Distance) :

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\big(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\big)$$

IS (Inception Score) :

$$\mathrm{IS} = \exp\big(\mathbb{E}_{x \sim p_g}\left[D_{\mathrm{KL}}(p(y|x) \| p(y))\right]\big)$$

Quand utiliser

FID (plus bas = meilleur) mesure la distance entre distributions réelles et générées dans l'espace des features Inception. Standard pour évaluer les GANs, diffusion models.

Comment optimiser

- **Wasserstein GAN** (WGAN-GP) : perte plus stable que GAN original.
- **Spectral Normalization** : stabiliser le discriminateur.
- **Progressive Growing** (StyleGAN) : entraîner à basse résolution d'abord.
- **Diffusion models** (DDPM, Stable Diffusion) : FID < 5 sur ImageNet vs ≈ 10 pour les meilleurs GANs.

Exemple illustratif

Comparaison sur ImageNet 256×256 :

Modèle	FID $\downarrow$	IS $\uparrow$
BigGAN	6.9	224
StyleGAN3	4.4	–
DiT-XL/2	**2.3**	**278**

Chapitre 8

Tableau de Bord — Synthèse Générale

Quelle métrique choisir selon le contexte ?

Domaine	Métrique principale	Métrique secondaire	Objectif typique
Régression	RMSE	R^2	RMSE↓, $R^2 \geq 0.9$
Classification équilibrée	Accuracy	AUC-ROC	Acc≥95%
Classification déséquilibrée	F1-Score	MCC, AUC-ROC	F1≥0.85
Détection d'objets	mAP@[.5:.95]	mAP@.5	mAP≥0.5
Segmentation	mIoU	Dice	mIoU≥0.75
Qualité image	SSIM	PSNR	SSIM≥0.9
Génération texte	BERTScore	BLEU, ROUGE-L	BERTScore≥0.85
LLM (benchmark)	MMLU	MT-Bench	MMLU≥70%
Code LLM	pass@1	pass@10	pass@1≥50%
LLM (alignement)	Win Rate	Reward Score	WR≥55% vs baseline
RAG Retrieval	Hit Rate@5	MRR, NDCG	HR@5≥0.85
RAG Generation	Faithfulness	Answer Relevancy	Faith≥0.90
AI Agent (tâche)	Task Success Rate	Step Efficiency	TSR≥50%
AI Agent (sécurité)	Hallucination Rate	Scope Creep	HallRate<1%
DL (classification)	Cross-Entropy Loss	Accuracy, F1	Loss↓, Val≈Train
DL (génération)	FID	IS, SSIM	FID<5
DL (entraînement)	Train/Val Loss Gap	Grad Norm	Gap≈0, MFU≥40%
Transformers	Perplexité	MFU	PPL↓, MFU≥40%

Règles d'or de l'évaluation IA

1. **Ne jamais utiliser une seule métrique** : toujours croiser au moins 2-3 métriques complémentaires.
2. **Valider sur un set de test jamais vu** pendant l'entraînement (stratified split, k-fold).
3. **Choisir la métrique selon l'impact business** : le coût d'un FN $\neq$ coût d'un FP.
4. **Surveiller la distribution** des données en production (data drift, concept drift).
5. **Évaluation humaine** obligatoire pour les LLMs et agents en production.
6. **Baseline simple d'abord** : comparer au modèle le plus simple possible.